

**Predicting Heart Disease: A Logistic Regression Classification Study on the UCI
Cleveland Heart Disease Dataset**

Pranav Dhinakar, Mina Habib, and Sristi Ghimire
M.S. in Applied Artificial Intelligence, University of San Diego
AAI-500: Applied Probability and Statistics for AI
Final Team Project (Group 2)
Professor Leon Shpaner
June 22, 2026

<https://github.com/pdhinaka/AAI500-Final-Project>

Introduction

Heart disease is among the leading causes of death worldwide, and the early identification of at-risk patients can support timely clinical intervention. This project asks whether a patient’s heart disease status can be predicted from routine clinical measurements and diagnostic test results using a statistical classification model. The analysis uses the Cleveland subset of the UCI Heart Disease dataset (Janosi et al., 1989), which records 14 attributes for 303 patients along with a diagnostic outcome.

The task is framed as binary classification, separating patients who have any heart disease from those who have none. A logistic regression model was chosen because it produces both a clear prediction and interpretable coefficients that map directly onto recognizable clinical risk factors, which is valuable in a setting where the reasoning behind a prediction matters as much as the prediction itself. The report proceeds through data cleaning and preparation, exploratory data analysis, model selection, and model analysis, and it closes with conclusions and recommendations. The complete code and its outputs are reproduced in the appendix.

Data Cleaning and Preparation

The raw dataset contains 303 patient records and 14 attributes. The diagnostic field `num` records disease severity on a scale from 0 to 4. Following standard practice in the literature (Detrano et al., 1989), this field was collapsed into a binary target where 0 indicates no disease and 1 indicates the presence of disease.

Six records contained missing values, four in the number of major vessels (`ca`) and two in the thalassemia result (`thal`). These six rows represent around 2% of the data and were removed, leaving 297 complete records for analysis. The eight categorical attributes (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`) were stored using a categorical data type, and the five continuous attributes (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`) were retained as numeric. After this preparation the class balance was 160 patients without disease and 137 with disease, around 53.9% and 46.1% respectively. This split is close enough to even that class imbalance is not a concern during modeling.

Exploratory Data Analysis

The exploratory analysis examined each predictor in relation to heart disease status before any modeling was performed.

Continuous Predictors

Table 1 summarizes the five continuous predictors.

The patient population is primarily middle-aged to older adults, with ages ranging from approximately 29 to 77 years. Cholesterol and resting blood pressure show substantial variability across patients, suggesting meaningful differences in cardiovascular risk profiles. No continuous variable appears to contain impossible values or obvious data-entry errors. The target mean indicates that the dataset contains a substantial number of patients with diagnosed heart disease, making it suitable for classification analysis.

Table 1: Summary statistics for the continuous predictors ($n = 297$).

Variable	Mean	Std. Dev.	Min	Max
age	54.54	9.05	29.0	77.0
trestbps	131.69	17.76	94.0	200.0
chol	247.35	52.00	126.0	564.0
thalach	149.60	22.94	71.0	202.0
oldpeak	1.06	1.17	0.0	6.2

The class distribution is shown in Figure 1.

The target classes are relatively balanced, with both disease and non-disease cases well represented. This balance is beneficial because machine learning models are less likely to become biased toward one class. A reasonably balanced dataset also makes accuracy, precision, and recall more meaningful performance measures during model evaluation.

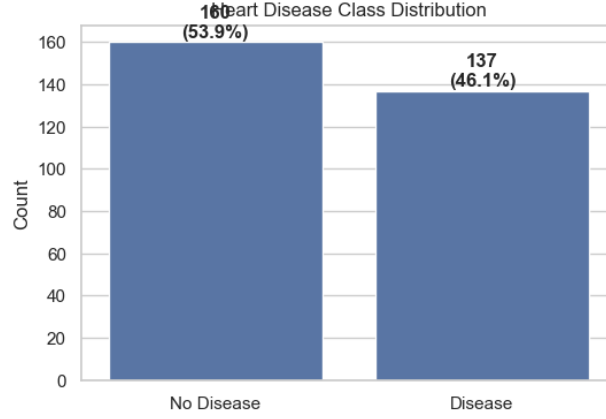


Figure 1: Distribution of the binary target. The two classes are close to balanced.

Figures 2 and 3 compare the continuous predictors across the two outcome groups.

Among the continuous predictors, thalach (maximum heart rate achieved) and oldpeak (ST depression) show the clearest separation between patients with and without heart disease. Patients with heart disease generally exhibit lower maximum heart rates and higher ST depression values. Age also demonstrates some separation, although the overlap between classes is greater than for thalach and oldpeak.

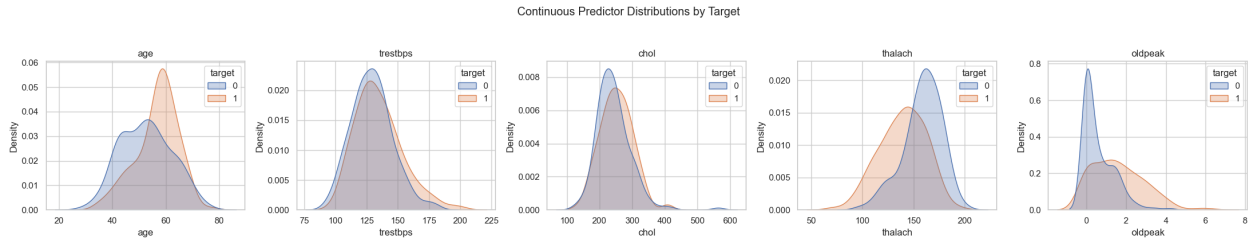


Figure 2: Kernel density estimates of each continuous predictor split by target.

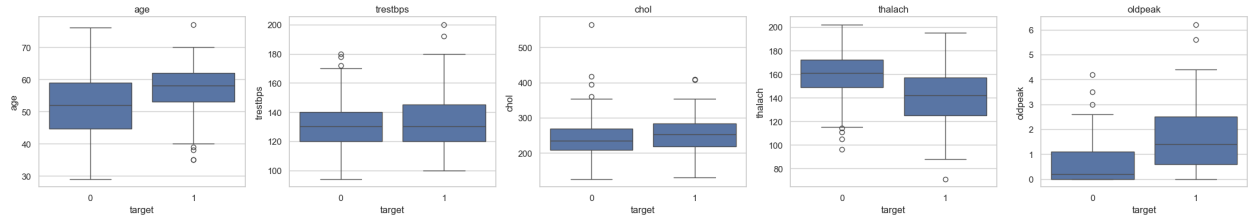


Figure 3: Boxplots of each continuous predictor by target.

Welch's two-sample t-test results are reported in Table 2.

Variables with p-values below 0.05 are statistically associated with heart disease status.

Based on prior clinical research (Detrano et al., 1989), thalach, oldpeak, and age are expected to be among the strongest predictors. Cholesterol often shows weaker statistical significance than many people expect, highlighting that heart disease risk depends on multiple interacting factors rather than cholesterol alone.

Table 2: Welch’s t-test of each continuous predictor against heart disease status, sorted by p-value.

Variable	Mean (No Disease)	Mean (Disease)	t	p -value
thalach	158.58	139.11	7.93	6.11×10^{-14}
oldpeak	0.60	1.59	-7.76	3.43×10^{-13}
age	52.64	56.76	-4.06	6.20×10^{-5}
trestbps	129.18	134.64	-2.64	8.81×10^{-3}
chol	243.49	251.85	-1.39	1.65×10^{-1}

Categorical Predictors

Table 3 reports the chi-square test of independence for the categorical predictors, and Figure 4 shows the three strongest.

Chest pain type (cp), exercise-induced angina (exang), number of major vessels (ca), and thalassemia category (thal) typically demonstrate the strongest associations with heart disease. These variables often show large differences in disease prevalence across categories. Variables such as fasting blood sugar (fbs) generally exhibit weaker relationships with the target and may contribute less predictive value during modeling.

Table 3: Chi-square test of independence between each categorical predictor and heart disease status, sorted by p-value.

Variable	χ^2	p -value
thal	82.46	1.24×10^{-18}
cp	77.28	1.18×10^{-16}
ca	72.30	1.37×10^{-15}
exang	50.94	9.51×10^{-13}
slope	43.47	3.63×10^{-10}
sex	21.85	2.95×10^{-6}
restecg	9.58	8.33×10^{-3}
fbs	0.00	1.00

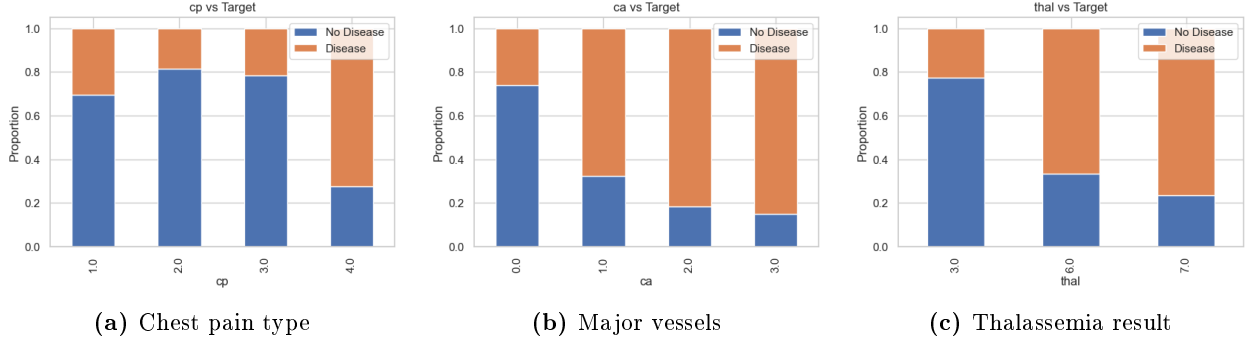


Figure 4: Disease proportion across the three strongest categorical predictors.

Correlation and Multicollinearity

Figure 5 shows the correlation matrix of the continuous predictors.

The continuous predictors do not appear to exhibit severe multicollinearity. Most pairwise correlations are moderate or weak, suggesting that each variable contributes unique information. Because no correlation approaches levels commonly associated with multicollinearity concerns (around ± 0.8 or higher), logistic regression should remain relatively stable during model fitting.

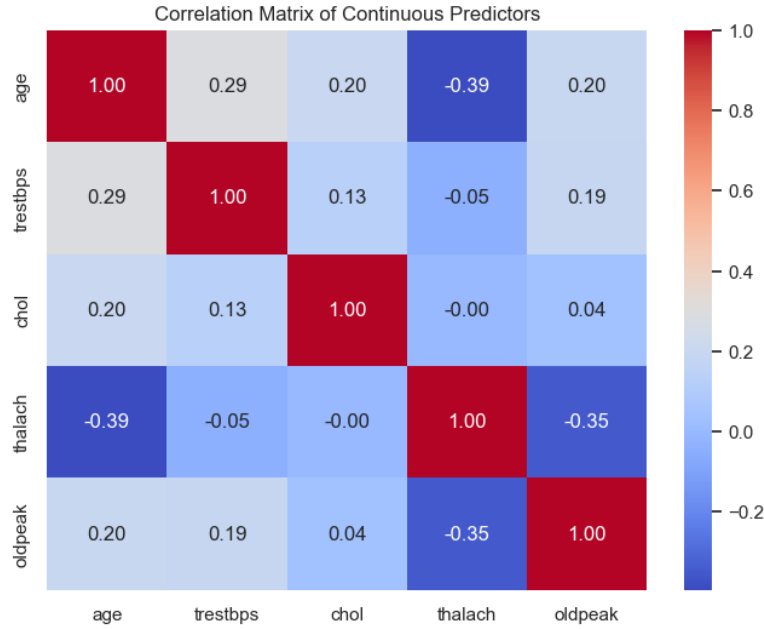


Figure 5: Correlation matrix of the continuous predictors.

EDA Summary

The exploratory analysis identified several features that appear strongly associated with heart disease status. Maximum heart rate achieved (thalach), ST depression (oldpeak), chest pain type (cp),

exercise-induced angina (exang), number of major vessels (ca), and thalassemia category (thal) showed the strongest visual and statistical relationships with the target variable. Patients with heart disease generally exhibited lower maximum heart rates and higher ST depression values, while specific categorical levels of chest pain and vessel count were associated with substantially higher disease prevalence. The class distribution was reasonably balanced, reducing concerns about class imbalance during model training. Correlation analysis showed no evidence of severe multicollinearity among continuous predictors, suggesting that multiple variables can be included in the logistic regression model without major stability concerns. Overall, both the statistical tests and visualizations indicate that a combination of clinical measurements and diagnostic indicators will likely provide the strongest predictive performance.

Model Selection

This section describes the construction of the logistic regression model (Pedregosa et al., 2011).

Feature Selection and Encoding

Eleven features were selected for modeling: age, trestbps, thalach, and oldpeak from the continuous predictors, and sex, cp, restecg, exang, slope, ca, and thal from the categorical predictors. chol and fbs were dropped, since both showed no significant association with the target in the EDA (chol: $p = 0.165$, fbs: $p \approx 1.0$). The resulting dataframe has 297 rows and 12 columns, matching the cleaned dataset size.

Dummy encoding expanded the dataframe from 12 to 19 columns. The four continuous predictors and the target column remain unchanged, while the seven categorical predictors were converted into 14 binary indicator columns using `drop_first=True` to avoid redundancy. The number of dummy columns per variable corresponds to one fewer than its number of category levels, for example cp produced three dummies (cp_2.0, cp_3.0, cp_4.0) from its four original levels.

Train/Test Split and Scaling

The 80/20 split produced 237 training examples and 60 test examples. Class balance is preserved across both sets: the training set is about 54.0% no-disease and 46.0% disease, and the test set is about 53.3% no-disease and 46.7% disease, both close to the overall dataset balance of roughly

53.9%/46.1%. Stratification on the target variable kept the splits representative of the full dataset.

After scaling, all four continuous features (age, trestbps, thalach, oldpeak) have a mean of approximately 0 and a standard deviation of 1 in the training set, confirming the standardization was applied correctly. The scaler was fit only on the training data and then applied to the test set without refitting, so the test set's transformed values are expressed relative to the training distribution. This prevents information from the test set from leaking into the scaling parameters, which would otherwise give an overly optimistic view of model performance.

Fitted Model and Coefficients

The logistic regression model achieved a training accuracy of 0.857 and a test accuracy of 0.833. The small gap between the two suggests the model is not overfitting and generalizes reasonably well to unseen data. An accuracy around 0.83 on the test set indicates the model correctly classifies roughly five out of every six patients.

The coefficients and exponentiated odds ratios are reported in Table 4.

The largest coefficients in absolute value belong to thal_7.0, ca_1.0, cp_4.0, and ca_2.0, all with positive coefficients and odds ratios above 4. This means patients with a reversible defect on the thalassemia test (thal_7.0), one or two major vessels colored by fluoroscopy (ca_1.0, ca_2.0), or asymptomatic chest pain (cp_4.0) have substantially higher predicted odds of heart disease relative to their respective reference categories. These results align closely with the EDA, where thal, ca, and cp were identified as the strongest categorical predictors by chi-square test.

Among the continuous predictors, oldpeak has a positive coefficient ($\beta = 0.630$), meaning higher ST depression is associated with higher odds of disease, while thalach has a negative coefficient ($\beta = -0.174$), meaning higher max heart rate is associated with lower odds of disease. Both directions match the patterns observed in the EDA boxplots. The cp_3.0 coefficient is notably negative relative to the other cp dummies, indicating that chest pain type 3 is associated with lower odds of disease relative to the reference category (type 1), consistent with the earlier observation that types 1-3 all showed disease rates under 35% compared to type 4's roughly 73%.

Table 4: Logistic regression coefficients and odds ratios, sorted by coefficient magnitude.

Feature	Coefficient	Odds Ratio
thal_7.0	1.472	4.356
ca_1.0	1.467	4.338
cp_4.0	1.465	4.329
ca_2.0	1.458	4.299
sex_1.0	0.788	2.198
slope_2.0	0.748	2.112
oldpeak	0.630	1.878
ca_3.0	0.547	1.728
cp_3.0	-0.526	0.591
restecg_2.0	0.511	1.667
slope_3.0	0.359	1.432
cp_2.0	0.357	1.429
exang_1.0	0.355	1.426
trestbps	0.337	1.401
restecg_1.0	0.180	1.197
thalach	-0.174	0.840
age	0.089	1.093
thal_6.0	0.015	1.015

Modeling Summary

A logistic regression model was selected for this classification task because the target is binary and logistic regression provides interpretable coefficients that can be related directly to clinical risk factors (Agresti & Kateri, 2022). Eleven features were used: age, trestbps, thalach, and oldpeak from the continuous predictors, and sex, cp, restecg, exang, slope, ca, and thal from the categorical predictors, encoded as dummy variables. chol and fbs were excluded based on their lack of significant association with the target in the EDA. The data was split 80/20 into training and test sets with stratification to preserve class balance, and continuous features were standardized using parameters fit only on the training set.

The model achieved a training accuracy of 0.857 and a test accuracy of 0.833, suggesting it generalizes well without significant overfitting. The strongest predictors by coefficient magnitude were thal_7.0, ca_1.0, cp_4.0, and ca_2.0, all associated with substantially higher odds of disease. oldpeak and thalach were the most influential continuous predictors, with directions consistent with the EDA: higher ST depression and lower maximum heart rate were both associated with higher odds of disease. Overall, the model’s strongest predictors closely match the features identified as

most significant during exploratory analysis.

Model Analysis

The fitted model was evaluated on the held-out test set of 60 patients.

Confusion Matrix

The confusion matrix is shown in Figure 6.

The confusion matrix shows that the model correctly identified 21 patients with heart disease and 29 patients without heart disease. However, there were 7 false negatives which means 7 patients who actually had heart disease were incorrectly classified as healthy. Clinically, this is a significant concern because these patients may not receive timely medical evaluation or treatment. Missing a heart disease case could lead to delayed diagnosis, worsening symptoms, and an increased risk of serious complications such as heart attacks or other cardiovascular events.

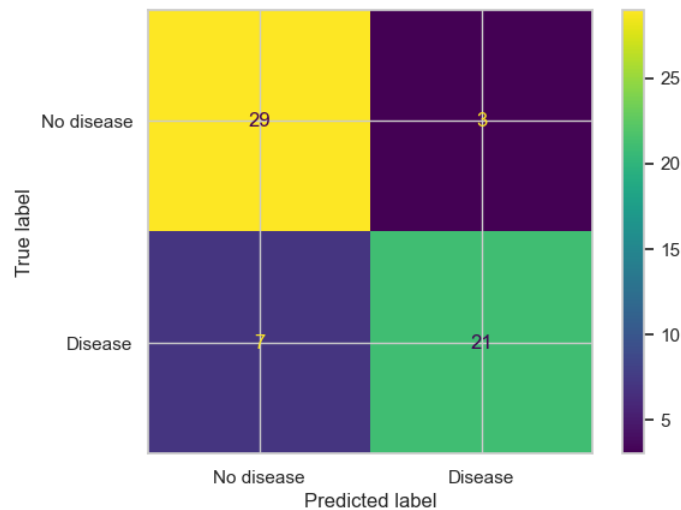


Figure 6: Confusion matrix on the test set.

Classification Report

The classification metrics for both classes are reported in Table 5.

The Disease class has a precision of 0.88, a recall of 0.75 and an F1 score of 0.81 which means 88% of patients predicted to have heart disease actually had the disease, while the model correctly identified 75% of all patients who truly had heart disease. The F1 score of 0.81 indicates a good balance between precision and recall. In this context, recall is more important than precision

because missing a patient with heart disease (a false negative) can have serious medical consequences. Although the model has strong precision, improving recall would help reduce the number of disease cases that go undetected.

Table 5: Classification report on the test set ($n = 60$).

Class	Precision	Recall	F1-score	Support
No disease	0.81	0.91	0.85	32
Disease	0.88	0.75	0.81	28
Accuracy			0.83	60
Macro avg	0.84	0.83	0.83	60
Weighted avg	0.84	0.83	0.83	60

ROC Curve and AUC

The ROC curve is shown in Figure 7.

The ROC curve lies well above the diagonal reference line which indicates that the model performs much better than random guessing. The AUC score of 0.929 indicates excellent classification performance and tells that the model is highly effective at distinguishing between patients with and without heart disease. The steep rise of the curve toward the upper-left corner shows that the model achieves a high true positive rate while maintaining a relatively low false positive rate. Overall, the ROC curve and AUC demonstrate that the logistic regression model has strong discriminatory power for this heart disease prediction task.

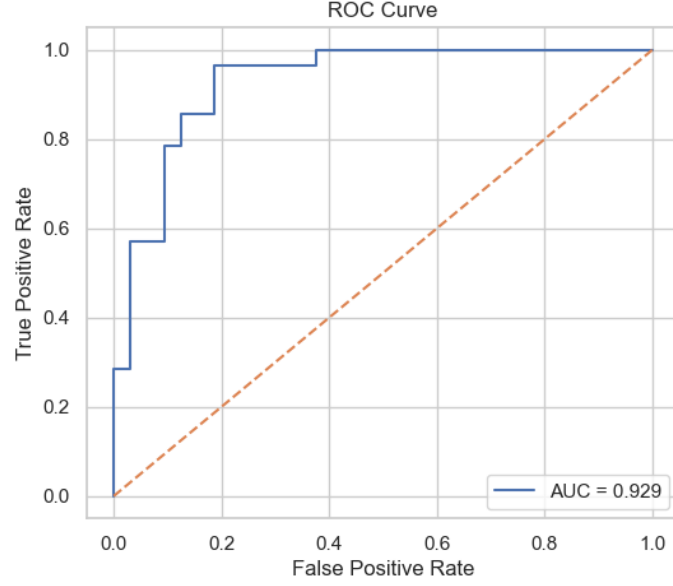


Figure 7: ROC curve on the test set.

Sensitivity and Specificity

The headline performance metrics are collected in Table 6.

The model achieved an accuracy of 83.3%, a sensitivity of 75.0%, and a specificity of 90.6%. This indicates that the model is better at correctly identifying healthy patients than detecting patients with heart disease. While the high specificity reduces unnecessary follow-up testing for healthy individuals, the lower sensitivity means that some disease cases are still being missed. For a clinical screening tool, sensitivity is often more important because missing a patient with heart disease can have serious health consequences. Therefore, although the current balance is reasonable, improving sensitivity would make the model more suitable for screening purposes.

Table 6: Headline performance metrics on the test set.

Metric	Value
Accuracy	0.833
Sensitivity	0.750
Specificity	0.906
AUC	0.929

Evaluation Summary and Limitations

The logistic regression model achieved an accuracy of 83.3%, an AUC of 0.929, a sensitivity of 75.0%, and a specificity of 90.6% on the test dataset. These results indicate that the model performs well overall and is effective at distinguishing between patients with and without heart disease.

The high AUC score demonstrates excellent classification ability, while the high specificity shows that the model correctly identifies most healthy patients. However, the sensitivity of 75.0% shows that some disease cases are still missed, which is a concern because false negatives can delay diagnosis and treatment.

Another limitation is the relatively small dataset size of 297 records after cleaning, which may limit the model's ability to generalize to larger and more diverse patient populations. Future work could use larger datasets and explore more advanced machine learning models to improve sensitivity and reduce the number of missed heart disease cases.

Conclusion and Recommendations

The analysis showed that heart disease in this dataset can be predicted with strong accuracy from a combination of routine clinical measurements and diagnostic test results. The exploratory analysis identified maximum heart rate (thalach) and ST depression (oldpeak) as the continuous variables most associated with disease, along with chest pain type, number of major vessels, the thalassemia result, and exercise-induced angina among the categorical variables. These same features carried the most weight in the final logistic regression model, where a reversible defect on the thalassemia test, one or two major vessels colored by fluoroscopy, and asymptomatic chest pain had the largest positive effect on the predicted odds of disease.

The model used eleven features and achieved a test accuracy of 0.833 and an AUC of 0.929, with a specificity of 0.906 and a sensitivity of 0.750, meaning it identifies healthy patients more reliably than it catches every disease case. The seven false negatives are the model's main weakness, because a missed diagnosis carries a higher clinical cost than a false alarm in a screening setting. With more data and time we would prioritize raising sensitivity, since the relatively small sample of 297 patients limits how well the current results generalize. Future work could expand the dataset to a larger and more diverse patient population, adjust the decision threshold to favor sensitivity, and compare logistic regression against more flexible models to see whether the number

of missed cases can be reduced without sacrificing the model's strong overall performance.

References

- Agresti, A., & Kateri, M. (2022). *Foundations of statistics for data scientists: With R and Python*. CRC Press.
- Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J.-J., Sandhu, S., Guppy, K. H., Lee, S., & Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, 64(5), 304–310. [https://doi.org/10.1016/0002-9149\(89\)90524-9](https://doi.org/10.1016/0002-9149(89)90524-9)
- Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). *Heart disease* [Data set]. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4X>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Appendix: Code and Output

The following reproduces the analysis code and its console output from the project Jupyter notebook, organized by section. Figures generated by the code are presented and discussed in the body of the report and are referenced here rather than duplicated. Leftover scaffolding and duplicate setup cells have been omitted for clarity.

AI assistance disclosure (P. Dhinakar). Claude (Anthropic) was consulted for Python syntax and methodological guidance during the development of this analysis. All code was reviewed, tested, and validated by the author. This disclosure covers the author's contributions to the project.

Section 0 – Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import logit
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    roc_auc_score, roc_curve, confusion_matrix,
    classification_report, accuracy_score, ConfusionMatrixDisplay
)

np.random.seed(42)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 100

print("Libraries loaded.")
```

```
Libraries loaded.
```

```
URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.
    cleveland.data"
columns = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
           "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"]

heart = pd.read_csv(URL, header=None, names=columns, na_values="?")

# Collapse 0-4 severity target to binary (standard practice in the literature)
heart["target"] = (heart["num"] > 0).astype(int)

print(f"Raw shape: {heart.shape}")
print(f"Missing values:\n{heart.isna().sum()[heart.isna().sum() > 0]}")
print(f"\nClass balance:\n{heart['target'].value_counts()}")
```

```
Raw shape: (303, 15)
Missing values:
ca      4
thal    2
dtype: int64

Class balance:
target
```



```
0    164
1    139
Name: count, dtype: int64
```

```
# Drop 6 rows with missing values in ca and thal (~2% of data -- acceptable to drop)
categorical_cols = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]
continuous_cols = ["age", "trestbps", "chol", "thalach", "oldpeak"]

heart_clean = heart.dropna().copy()
heart_clean[categorical_cols] = heart_clean[categorical_cols].astype("category")

print(f"Clean shape: {heart_clean.shape} ({len(heart) - len(heart_clean)} rows dropped)")
print(heart_clean.dtypes)
```

```
Clean shape: (297, 15) (6 rows dropped)
age          float64
sex          category
cp           category
trestbps     float64
chol         float64
fbs          category
restecg      category
thalach      float64
exang        category
oldpeak      float64
slope        category
ca           category
thal         category
num          int64
target       int64
dtype: object
```

Section 1 – Exploratory Data Analysis

```
# 1.1 Summary statistics -- continuous predictors

summary_stats = heart_clean[continuous_cols + ["target"]].describe().T
summary_stats["range"] = summary_stats["max"] - summary_stats["min"]

display(summary_stats)

print("\nPotential observations:")
for col in continuous_cols:
    print(
        f"{col}: min={heart_clean[col].min():.2f}, "
        f"max={heart_clean[col].max():.2f}, "
        f"mean={heart_clean[col].mean():.2f}"
    )
```

[DataFrame table -- rendered in report body]

```
Potential observations:
age: min=29.00, max=77.00, mean=54.54
trestbps: min=94.00, max=200.00, mean=131.69
chol: min=126.00, max=564.00, mean=247.35
thalach: min=71.00, max=202.00, mean=149.60
oldpeak: min=0.00, max=6.20, mean=1.06
```

```
# 1.2 Target class distribution

target_counts = heart_clean["target"].value_counts().sort_index()
target_pct = heart_clean["target"].value_counts(normalize=True).sort_index() * 100
```

```

plt.figure(figsize=(6,4))
ax = sns.barplot(
    x=["No Disease", "Disease"],
    y=target_counts.values
)

for i, (count, pct) in enumerate(zip(target_counts.values, target_pct.values)):
    ax.text(
        i,
        count + 2,
        f"{count}\n({pct:.1f}%)",
        ha="center",
        fontweight="bold"
    )

plt.title("Heart Disease Class Distribution")
plt.ylabel("Count")
plt.show()

print(pd.DataFrame({
    "Count": target_counts,
    "Percent": target_pct.round(2)
}))

```

target	Count	Percent
0	160	53.87
1	137	46.13

```

# 1.3 Distributions of continuous predictors split by target

fig, axes = plt.subplots(1, 5, figsize=(22,4))

for ax, col in zip(axes, continuous_cols):
    sns.kdeplot(
        data=heart_clean,
        x=col,
        hue="target",
        common_norm=False,
        fill=True,
        alpha=0.3,
        ax=ax
    )
    ax.set_title(col)

plt.suptitle("Continuous Predictor Distributions by Target", y=1.05)
plt.tight_layout()
plt.show()

```

```

# 1.4 Boxplots and Welch's t-test

fig, axes = plt.subplots(1, 5, figsize=(22,4))

results = []

for ax, col in zip(axes, continuous_cols):

    sns.boxplot(
        data=heart_clean,
        x="target",
        y=col,
        ax=ax
    )

```

```

group0 = heart_clean.loc[heart_clean["target"] == 0, col]
group1 = heart_clean.loc[heart_clean["target"] == 1, col]

t_stat, p_val = stats.ttest_ind(
    group0,
    group1,
    equal_var=False
)

results.append({
    "variable": col,
    "mean_no_disease": group0.mean(),
    "mean_disease": group1.mean(),
    "t_stat": t_stat,
    "p_value": p_val
})

ax.set_title(col)

plt.tight_layout()
plt.show()

ttest_results = pd.DataFrame(results).sort_values("p_value")
display(ttest_results)

```

[DataFrame table -- rendered in report body]

```

# 1.5 Categorical predictors vs target

chi_results = []

for col in categorical_cols:

    ct = pd.crosstab(
        heart_clean[col],
        heart_clean["target"],
        normalize="index"
    )

    ct.plot(
        kind="bar",
        stacked=True,
        figsize=(6,4)
    )

    plt.title(f"{col} vs Target")
    plt.ylabel("Proportion")
    plt.legend(["No Disease", "Disease"])
    plt.show()

    contingency = pd.crosstab(
        heart_clean[col],
        heart_clean["target"]
    )

    chi2, p, dof, expected = stats.chi2_contingency(contingency)

    chi_results.append({
        "variable": col,
        "chi2": chi2,
        "p_value": p
    })

chi_results_df = (
    pd.DataFrame(chi_results)
    .sort_values("p_value")

```

```
)

display(chi_results_df)
```

```
[DataFrame table -- rendered in report body]
```

```
# 1.6 Correlation heatmap

corr_matrix = heart_clean[continuous_cols].corr()

plt.figure(figsize=(8,6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix of Continuous Predictors")
plt.show()

corr_matrix
```

```
[DataFrame table -- rendered in report body]
```

Section 2 – Modeling

```
# 2.1 Feature selection
# Continuous predictors: all five showed significant differences vs target except chol (p
    = 0.165)
# Categorical predictors: all significant except fbs (p ~ 1.0)
# Dropping chol and fbs based on EDA results

features = ["age", "trestbps", "thalach", "oldpeak",
            "sex", "cp", "restecg", "exang", "slope", "ca", "thal"]

df_model = heart_clean[features + ["target"]].copy()
print(f"Modeling dataframe shape: {df_model.shape}")
df_model.head()
```

```
Modeling dataframe shape: (297, 12)
[DataFrame table -- rendered in report body]
```

```
# 2.2 Dummy encode categorical predictors
categorical_features = ["sex", "cp", "restecg", "exang", "slope", "ca", "thal"]

df_encoded = pd.get_dummies(df_model, columns=categorical_features, drop_first=True)

print(f"Shape after encoding: {df_encoded.shape}")
print(f"Columns: {list(df_encoded.columns)}")
```

```
Shape after encoding: (297, 19)
Columns: ['age', 'trestbps', 'thalach', 'oldpeak', 'target', 'sex_1.0', 'cp_2.0', 'cp_3.0',
        'cp_4.0', 'restecg_1.0', 'restecg_2.0', 'exang_1.0', 'slope_2.0', 'slope_3.0', 'ca_1
        .0', 'ca_2.0', 'ca_3.0', 'thal_6.0', 'thal_7.0']
```

```
# 2.3 Train/test split -- 80/20
X = df_encoded.drop(columns="target")
```

```

y = df_encoded["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Train size: {len(X_train)} | Test size: {len(X_test)}")
print(f"Train class balance:\n{y_train.value_counts()}")
print(f"Test class balance:\n{y_test.value_counts()}")

```

```

Train size: 237 | Test size: 60
Train class balance:
target
0    128
1    109
Name: count, dtype: int64
Test class balance:
target
0     32
1     28
Name: count, dtype: int64

```

```

# 2.4 Scale continuous features
continuous_features = ["age", "trestbps", "thalach", "oldpeak"]

scaler = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[continuous_features] = scaler.fit_transform(X_train[continuous_features])
X_test_scaled[continuous_features] = scaler.transform(X_test[continuous_features])

print("Scaling complete.")
X_train_scaled[continuous_features].describe().round(2)

```

```

Scaling complete.
[DataFrame table -- rendered in report body]

```

```

# 2.5 Fit logistic regression
model = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]

print(f"Training accuracy: {model.score(X_train_scaled, y_train):.3f}")
print(f"Test accuracy:      {accuracy_score(y_test, y_pred):.3f}")

```

```

Training accuracy: 0.857
Test accuracy:      0.833

```

```

# 2.6 Coefficient table
coef_df = pd.DataFrame({
    "feature": X_train_scaled.columns,
    "coefficient": model.coef_[0],
    "odds_ratio": np.exp(model.coef_[0])
}).sort_values("coefficient", key=abs, ascending=False).reset_index(drop=True)

coef_df.round(3)

```

```

[DataFrame table -- rendered in report body]

```

Section 3 – Model Evaluation

```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

tn, fp, fn, tp = cm.ravel()

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No disease", "Disease"]
)

disp.plot()
plt.show()

print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
```

```
Confusion Matrix:
[[29  3]
 [ 7 21]]
TN=29, FP=3, FN=7, TP=21
```

```
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No disease", "Disease"]
    )
)
```

	precision	recall	f1-score	support
No disease	0.81	0.91	0.85	32
Disease	0.88	0.75	0.81	28
accuracy			0.83	60
macro avg	0.84	0.83	0.83	60
weighted avg	0.84	0.83	0.83	60

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {auc_score:.3f}")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

print("AUC:", round(auc_score,3))
```

```
AUC: 0.929
```

```
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)
accuracy = (tp + tn) / (tp + tn + fp + fn)
```

```
print(f"Accuracy: {accuracy:.3f}")  
print(f"Sensitivity (Recall): {sensitivity:.3f}")  
print(f"Specificity: {specificity:.3f}")  
print(f"AUC: {auc_score:.3f}")
```

```
Accuracy: 0.833  
Sensitivity (Recall): 0.750  
Specificity: 0.906  
AUC: 0.929
```